



Rybina search (rybina)

En la Schueberfouer, la mayor feria de Luxemburgo, hay un popular juego de adivinar números. El feriante, Rybina, esconde una contraseña secreta de la forma AB , donde A y B son strings de N dígitos. Rybina te permite intentar adivinar la contraseña y, después de cada intento, te dirá si tu respuesta es demasiado alta o demasiado baja.

Sin embargo, después de cada intento, Rybina podría intercambiar las dos mitades de la contraseña, es decir, AB se convierte en BA (o BA se convierte en AB). También te dirá si lo ha hecho. Tu siguiente intento se comparará entonces con la contraseña intercambiada.

Tu tarea es descubrir la contraseña secreta usando como máximo 64 intentos.

Implementación

Tendrás que enviar un único archivo fuente `.cpp`.



Entre los adjuntos de esta tarea encontrarás una plantilla `rybina.cpp` con una implementación de ejemplo.

Tendrás que implementar la siguiente función:

C++	<code>void rybina_search(int N)</code>
-----	--

- El entero N representa el número de dígitos en cada mitad de la contraseña.

Tu programa puede llamar a la siguiente función, que ya está definida en el grader:

C++	<code>GuessResult guess(long long X)</code>
-----	---

- El entero X representa la contraseña que introduces como intento. Debe ser no negativo y debe contener como máximo $2N$ dígitos al escribirse en decimal (es decir, $0 \leq X < 10^{2N}$).
- Si tu intento es correcto, tu programa termina y recibes el veredicto `accepted` para el caso de prueba actual.
- En caso contrario, la función devuelve un `GuessResult`, una struct con un valor entero `comparison` y un valor booleano `swapped`.
- El valor de `comparison` es 1 cuando tu respuesta (X) era demasiado baja y -1 si tu respuesta era demasiado alta.
- El valor de `swapped` indica si la contraseña se ha intercambiado después de que hicieras tu intento.

Ten en cuenta que el interactor es **adaptativo** y que, por tanto, la contraseña correcta o los intercambios para un caso de prueba no están fijados de antemano.

Sample Grader

Entre los adjuntos de esta tarea encontrarás una versión simplificada del grader utilizado durante la evaluación, que puedes usar para probar tus soluciones localmente. El grader de ejemplo lee datos de `stdin` y llama a la función `rybina_search`, usando el siguiente formato de entrada.

La entrada consta de dos líneas:

- Línea 1: el entero N .
- Línea 2: dos enteros S y T . El primero, S , es la semilla usada para decidir los intercambios aleatorios; el segundo, T , es el objetivo: la contraseña inicial AB escrita como un único entero con $0 \leq T < 10^{2N}$.

Después de cada intento, el grader de ejemplo decide al azar si intercambiar las dos mitades de la contraseña, usando la semilla S . (El interactor usado durante la evaluación oficial, en cambio, es adaptativo.)

El grader de ejemplo informa del resultado en `stderr`: si tu solución determina correctamente la contraseña imprime el número de intentos usados y, en caso contrario, imprime un breve mensaje con la explicación (un intento inválido, o demasiados intentos).

Restricciones

- $1 \leq N \leq 9$.

Puedes llamar a la función `guess` como máximo 64 veces.

Puntuación

Tu programa será evaluado con varios casos de prueba agrupados en subtareas. Para obtener la puntuación de una subtarea, tu programa debe resolver correctamente todos sus casos de prueba.

- **Subtask 0 [0 puntos]**: Casos de prueba de ejemplo.
- **Subtask 1 [5 puntos]**: $N = 1$ y $B = 0$, es decir, la segunda mitad de la contraseña inicial es 0.
- **Subtask 2 [5 puntos]**: $N = 1$.
- **Subtask 3 [10 puntos]**: $N \leq 4$ y Rybina nunca intercambiará la contraseña.
- **Subtask 4 [20 puntos]**: Rybina nunca intercambiará la contraseña.
- **Subtask 5 [15 puntos]**: Rybina siempre intercambiará la contraseña.
- **Subtask 6 [20 puntos]**: $N \leq 4$
- **Subtask 7 [25 puntos]**: Sin restricciones adicionales.

Ejemplos de entrada/salida

Grader	Solución
<code>rybina_search(2)</code>	
	<code>guess(1000)</code>
<code>return [1, 1]</code>	
	<code>guess(1000)</code>
<code>return [-1, 0]</code>	
	<code>guess(412)</code>
<code>return [0, 0]</code>	

Explicación

En el primer ejemplo, $N = 2$ y las dos partes de la contraseña secreta son 12 y 04. Se da la respuesta 1000; en este momento la contraseña es 1204, por lo que la respuesta es demasiado baja y, después del intento, Rybina informa de que la contraseña se ha intercambiado. La contraseña es ahora 412 y se hace el mismo intento con 1000. Esta vez la respuesta es demasiado alta y Rybina informa de que la contraseña no se ha intercambiado después. Finalmente, se intenta la contraseña 412, que resulta ser correcta, y el programa termina.